

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("carbon_emission_dataset_with_Industry.csv")
```

```
print("First 5 Rows:")
print(df.head())
```

```
print("\nDataset Shape:")
print(df.shape)
```

```
print("\nColumn Names:")
print(df.columns)
```

```
print("\nDataset Info:")
print(df.info())
```

```
print("\nStatistical Summary:")
print(df.describe())
```

```
# Check Missing Values
```

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

```
print("\nMissing Value Percentage:")
```

```
print((df.isnull().sum() / len(df)) * 100)
```

```
print("\nDuplicate Rows:")
```

```
print(df.duplicated().sum())
```

```
df.drop_duplicates(inplace=True)
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
print("\nDataset Info After Cleaning:")
```

```
print(df.info())
```

```
print("\nFinal Dataset Shape:")
```

```
print(df.shape)
```

```
columns = [  
    'Total_Energy_Consumption_kWh',  
    'Renewable_Energy_Consumption_kWh',  
    'NonRenewable_Energy_Consumption_kWh',  
    'Production_Output_Units',  
    'Supply_Chain_Transport_km'  
]
```

```
for col in columns:  
    print(f"\nStatistics for {col}")  
    print("Mean:", df[col].mean())  
    print("Median:", df[col].median())  
    print("Max:", df[col].max())  
    print("Min:", df[col].min())
```

```
for col in columns:  
    plt.figure(figsize=(8,5))  
    sns.histplot(df[col], kde=True)  
    plt.title(f"Distribution of {col}")  
    plt.show()
```

```
for col in columns:  
    plt.figure(figsize=(8,5))  
    bins = pd.cut(df[col], bins=10)  
    bins.value_counts().sort_index().plot(kind='bar')  
    plt.title(f"Frequency of {col}")
```

```
plt.xlabel(col)
plt.ylabel("Frequency")
plt.show()
```

```
plt.figure(figsize=(12,6))
```

```
plt.plot(
    df['Date'],
    df['Total_Energy_Consumption_kWh'],
    label='Total Energy'
)
```

```
plt.plot(
    df['Date'],
    df['Renewable_Energy_Consumption_kWh'],
    label='Renewable Energy'
)
```

```
plt.plot(
    df['Date'],
    df['NonRenewable_Energy_Consumption_kWh'],
    label='Non-Renewable Energy'
)
```

```
plt.title("Energy Consumption Over Time")
plt.xlabel("Date")
```

```
plt.ylabel("kWh")
```

```
plt.legend()
```

```
plt.show()
```

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(
```

```
    x='Temperature_C',
```

```
    y='Carbon_Emission_tCO2e_TARGET',
```

```
    data=df
```

```
)
```

```
plt.title("Carbon Emissions vs Temperature")
```

```
plt.show()
```

```
plt.figure(figsize=(10,5))
```

```
sns.scatterplot(
```

```
    x='Date',
```

```
    y='Carbon_Emission_tCO2e_TARGET',
```

```
    data=df
```

```
)
```

```
plt.title("Carbon Emissions vs Time")
```

```
plt.show()
```

```
plt.figure(figsize=(12,6))  
plt.plot(df['Date'], df['Total_Energy_Consumption_kWh'], label='Total Energy')  
plt.plot(df['Date'], df['Renewable_Energy_Consumption_kWh'], label='Renewable Energy')  
plt.plot(df['Date'], df['NonRenewable_Energy_Consumption_kWh'], label='Non-Renewable  
Energy')  
plt.title("Energy Consumption Over Time")  
plt.xlabel("Date")  
plt.ylabel("kWh")  
plt.legend()  
plt.show()
```

```
plt.figure(figsize=(8,5))  
sns.scatterplot(x='Temperature_C', y='Carbon_Emission_tCO2e_TARGET', data=df)  
plt.title("Carbon Emissions vs Temperature")  
plt.show()
```

```
plt.figure(figsize=(10,5))  
sns.scatterplot(x='Date', y='Carbon_Emission_tCO2e_TARGET', data=df)  
plt.title("Carbon Emissions vs Time")  
plt.show()
```

```
plt.figure(figsize=(10, 8))  
corr = df.corr(numeric_only=True)  
sns.heatmap(corr, annot=True, cmap='coolwarm')  
plt.title("Correlation Heatmap")
```

```
plt.show()
```

```
plt.figure(figsize=(12, 6))
```

```
sns.boxplot(x='Industry_Sectors', y='Carbon_Emission_tCO2e_TARGET', data=df)
```

```
plt.title("Carbon Emissions by Industry Sectors")
```

```
plt.xticks(rotation=45)
```

```
plt.show()
```

```
features = ['Carbon_Emission_tCO2e_TARGET', 'Total_Energy_Consumption_kWh',  
'Production_Output_Units', 'Temperature_C']
```

```
sns.pairplot(df[features])
```

```
plt.show()
```

```
plt.figure(figsize=(10, 5))
```

```
df.groupby('Region')['Carbon_Emission_tCO2e_TARGET'].mean().plot(kind='bar')
```

```
plt.title("Average Carbon Emissions by Region")
```

```
plt.show()
```

#5. Time Series Exploration

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from statsmodels.tsa.seasonal import seasonal_decompose
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
df2 = df.sort_values('Date').set_index('Date')
```

```
plt.figure(figsize=(15, 3))
```

```
plt.plot(df2.index, df2['Carbon_Emission_tCO2e_TARGET'], color='r')
```

```
plt.title("Emissions Over Time")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Carbon Emission")
```

```
plt.show()
```

```
plt.figure(figsize=(15, 3))
```

```
plt.plot(df2.index, df2['Temperature_C'], color='b')
```

```
plt.title("Temperature Over Time")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Temperature")
```

```
plt.show()
```

```
n = 60
```

```
m_avg = df2['Carbon_Emission_tCO2e_TARGET'].rolling(n).mean()
```

```
m_var = df2['Carbon_Emission_tCO2e_TARGET'].rolling(n).var()
```

```
plt.figure(figsize=(12, 5))
```

```
plt.plot(df2['Carbon_Emission_tCO2e_TARGET'], alpha=0.4, color='gray')
```

```
plt.plot(m_avg, color='blue', label='Moving Average')
```

```
plt.plot(m_var, color='green', label='Rolling Variance')
```

```
plt.title("Rolling Statistics (60 Days)")
```

```
plt.legend()
```



```
plt.show()
```

```
monthly_data = df2['Carbon_Emission_tCO2e_TARGET'].resample('M').mean().ffill()
```

```
try:
```

```
    res = seasonal_decompose(monthly_data, model='additive', period=12)
```

```
    fig, (S1, S2, S3, S4) = plt.subplots(4, 1, figsize=(12, 8), sharex=True)
```

```
    res.observed.plot(ax=S1, color='k')
```

```
    S1.set_ylabel('Observed')
```

```
    res.trend.plot(ax=S2, color='b')
```

```
    S2.set_ylabel('Trend')
```

```
    res.seasonal.plot(ax=S3, color='g')
```

```
    S3.set_ylabel('Seasonal')
```

```
    res.resid.plot(ax=S4, color='r', style=':')
```

```
    S4.set_ylabel('Residuals')
```

```
    plt.show()
```

```
except:
```

```
    pass
```